

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №6

Тема: Патерни проектування

Виконав:
студент групи ІА-34
Єресько І.С
Перевірив:
Мягкий М.Ю

Київ-2025

Короткі теоретичні відомості

Шаблон «Abstract Factory» використовується для створення груп пов'язаних між собою об'єктів без прив'язки до їхніх конкретних класів. Ідея полягає в тому, що існує спільний інтерфейс фабрики, а під кожне сімейство продуктів створюється окрема реалізація цього інтерфейсу. Це дозволяє легко замінювати одне сімейство іншим та гарантувати, що всі створені об'єкти будуть узгоджені між собою. Проте недоліком є складність розширення: якщо потрібно додати новий тип продукту, необхідно змінювати всі фабрики.

У задачі генерації кімнат для roguelike-гри це дає змогу створювати стіни, двері, меблі та інші елементи в одному стилі - хайтек, модерн чи класичний офіс. Кожен тип елементів має свій інтерфейс, а під кожен стиль - свою реалізацію, наприклад HighTechTable або ModernWindow. Абстрактна фабрика визначає методи для створення всіх елементів кімнати, а конкретні фабрики створюють повний набір продуктів одного стилю. Генератор отримує лише фабрику й таким чином гарантовано створює кімнату в єдиному стилі, працюючи через інтерфейси, а не конкретні типи. Додавання нового стилю зводиться до створення нових класів продуктів та окремої фабрики для них. Хоча така структура робить код більш організованим і забезпечує узгодженість елементів, вона збільшує складність проекту та ускладнює додавання нових типів продуктів.

Шаблон «Factory Method» визначає інтерфейс для створення об'єктів певного базового типу й дозволяє підмінювати їхні конкретні реалізації. Завдяки цьому фабричний метод часто називають «віртуальним конструктором». Загальний принцип полягає в тому, що частина поведінки виноситься в базовий клас, а створення різних підтипів делегується дочірнім класам. Наприклад, у системі роботи з мережевими пакетами існує базовий клас Packet і підкласи TcpPacket та UdpPacket. Під них створюються відповідні творці - TcpCreator і UdpCreator, фабричний метод яких повертає потрібний тип пакету. Загальна логіка взаємодії з пакетами залишається в базовому PacketCreator. Такий підхід спрощує додавання нових типів пакетів і зменшує залежність від конкретних класів, але водночас може призвести до збільшення кількості ієрархій.

Шаблон «Memento» дає можливість зберігати і відновлювати стан об'єктів без порушення інкапсуляції. Об'єкт Memento містить знімок стану, але доступ до цього стану має лише початковий об'єкт Originator. Передача та зберігання знімків доручається Caretaker, який не має можливості читати або змінювати їхній вміст. Такий механізм позбавляє необхідності зберігати історію змін всередині самого об'єкта та спрощує його структуру. Однак часте створення знімків може суттєво збільшити споживання пам'яті, особливо якщо старі мemento не видаляються.

Шаблон «Decorator» дає можливість гнучко та динамічно додавати нові функції окремим об'єктам під час роботи програми. Він «обгортає» вихідний об'єкт, зберігаючи його поведінку, але розширюючи її додатковими діями. Це набагато гнучкіше, ніж створення численних підкласів, адже зміни застосовуються лише до тих об'єктів, які цього потребують. У прикладі з формуванням контекстного меню кожен елемент має метод UpdateContextMenu() і посилання на «батьківський» елемент. Компонент може додати свої пункти меню і передати обробку далі по ланцюжку. Якщо ж елемент не має нічого додати, він просто викликає метод у наступного. Таке рішення мінімізує залежність між об'єктами та дозволяє легко змінювати склад декораторів, хоча інколи запит може так і не бути оброблений.

Шаблон «Observer» забезпечує механізм повідомлення підписників про зміну стану об'єкта. Він встановлює зв'язок «один-до-багатьох», коли суб'єкт надсилає сповіщення всім спостерігачам, які можуть реагувати на зміни й оновлювати власний стан. Наприклад, у банківській системі кілька користувачів можуть одночасно переглядати баланс клієнта, і коли баланс змінюється, кожен з них автоматично бачить оновлені дані. Такий підхід дозволяє працювати асинхронно й підтримує слабкий зв'язок між об'єктами, даючи можливість вільно додавати або прибирати спостерігачів у будь-який момент. Недоліком є те, що порядок сповіщення не гарантується, що може бути критичним у деяких системах.

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator» та навчитися застосовувати їх в реалізації програмної системи.

Хід роботи

8. Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server)

Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або одночасно шляхом розділення вікна).

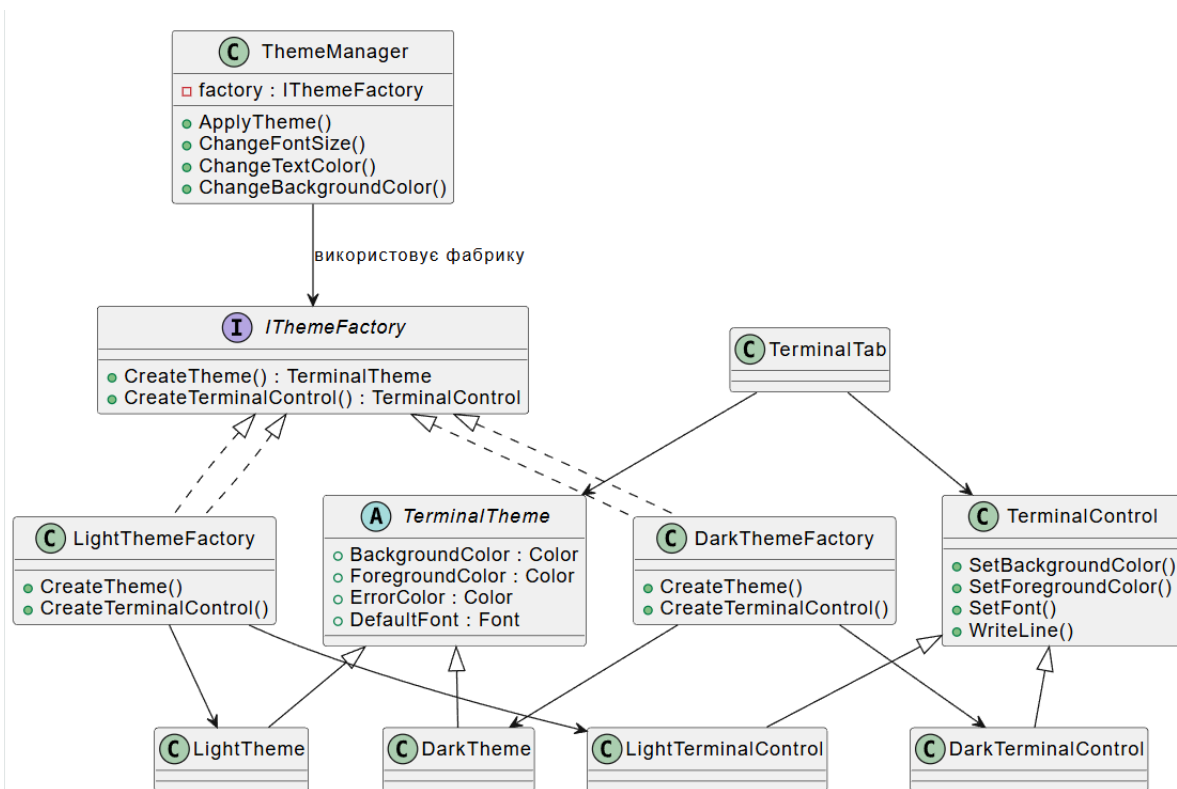


Рис 1: ClassDiagram (abstract factory)

[IThemeFactory](#)

[TerminalControl](#)

[TerminalTheme](#)

[LightTerminalControl](#)

[DarkTerminalControl](#)

[LightThemeFactory](#)

[ThemeManager](#)

Діаграма показує, як у системі TerminalApp використовується шаблон **Abstract Factory** для створення цілісних сімейств елементів інтерфейсу, що відповідають певній темі. У центрі цієї архітектури знаходиться інтерфейс **IThemeFactory**, який визначає два методи: створення об'єкта теми (**TerminalTheme**) і створення елемента керування терміналом (**TerminalControl**). Це означає, що клієнтський код не створює теми безпосередньо, а завжди працює через фабрику, яка гарантує узгодженість компонентів.

Абстрактний клас **TerminalTheme** описує властивості теми, такі як кольори фону, тексту, помилок і шрифт. Він виступає спільним інтерфейсом для конкретних тем, наприклад Light і Dark. Клас **TerminalControl** представляє компонент керування терміналом, який уміє змінювати кольори, шрифт та відображати текст. Він також має конкретні реалізації для кожної теми, щоб зовнішній вигляд елементів інтерфейсу був повністю узгодженим.

У діаграмі є дві повноцінні реалізації сімейств продуктів: **LightTheme + LightTerminalControl** та **DarkTheme + DarkTerminalControl**. Вони відповідають двом конкретним варіантам оформлення -світлому і темному. Для кожного сімейства існує своя фабрика: **LightThemeFactory** та **DarkThemeFactory**, які реалізують інтерфейс IThemeFactory і повертають відповідні продукти своєї теми. Це забезпечує чіткий поділ між логікою створення об'єктів та їх використанням.

Клас **ThemeManager** виступає клієнтом шаблону. Він зберігає посилання на активну фабрику і через неї отримує об'єкти теми та елементи керування. Це дозволяє йому застосовувати тему, змінювати параметри

відображення або шрифти, не знаючи, які саме класи створюються під капотом. Усе, що потрібно ThemeManager - це знати, яку фабрику використовувати в конкретний момент. Заміна фабрики одразу призводить до створення нового сімейства відповідних об'єктів.

Висновок:

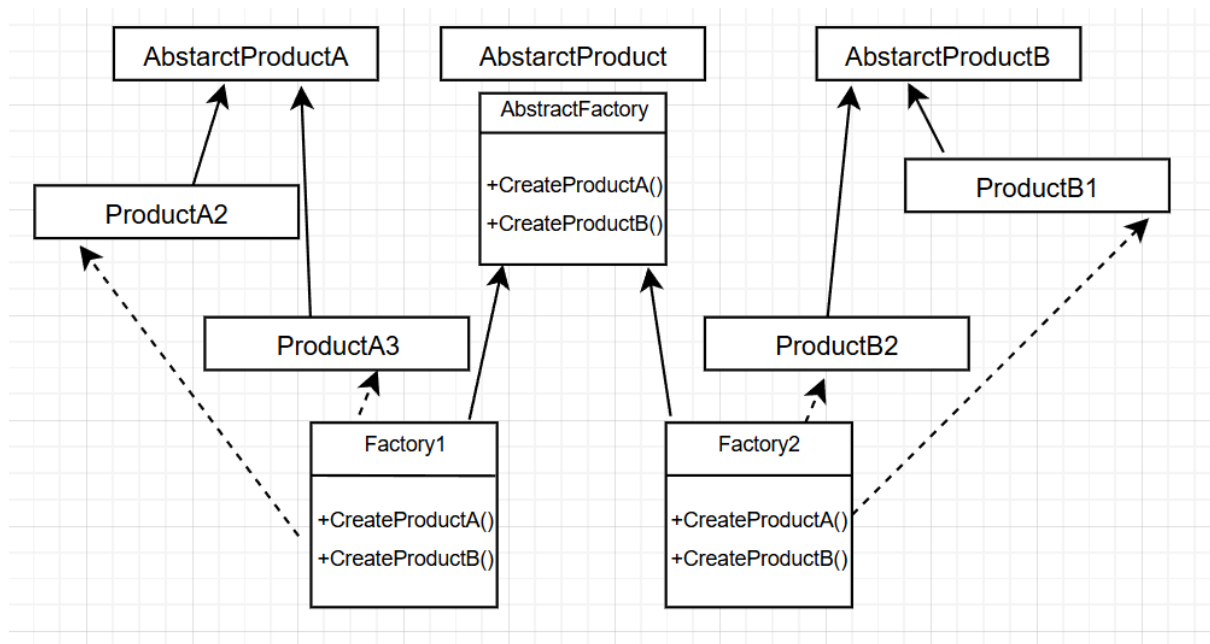
У межах виконання роботи я ознайомився з ключовими патернами проектування, зокрема «Abstract Factory», «Factory Method», «Memento», «Observer» та «Decorator». Було розглянуто їхню структуру, призначення, переваги та сфери застосування. Особливу увагу приділено патерну Abstract Factory, який я використав у реалізації програмної системи для створення узгоджених сімейств об'єктів – тем оформлення та елементів керування терміналом. Це дозволило забезпечити гнучкість системи, спростити зміну варіантів інтерфейсу та уникнути жорсткого зв'язування між компонентами. Ознайомлення з патернами та їх практичне застосування сприяло кращому розумінню принципів модульності, розширюваності та підтримуваності програмного коду.

Контрольні запитання

1. Яке призначення шаблону «Абстрактна фабрика»?

Шаблон дає можливість спростити та відокремити процес створення цілих сімейств пов'язаних об'єктів на стороні клієнта. Завдяки цьому клієнтська частина працює тільки з абстракціями, а конкретні об'єкти створюються приховано та узгоджено між собою.

2. Нарисуйте структуру шаблону «Абстрактна фабрика».



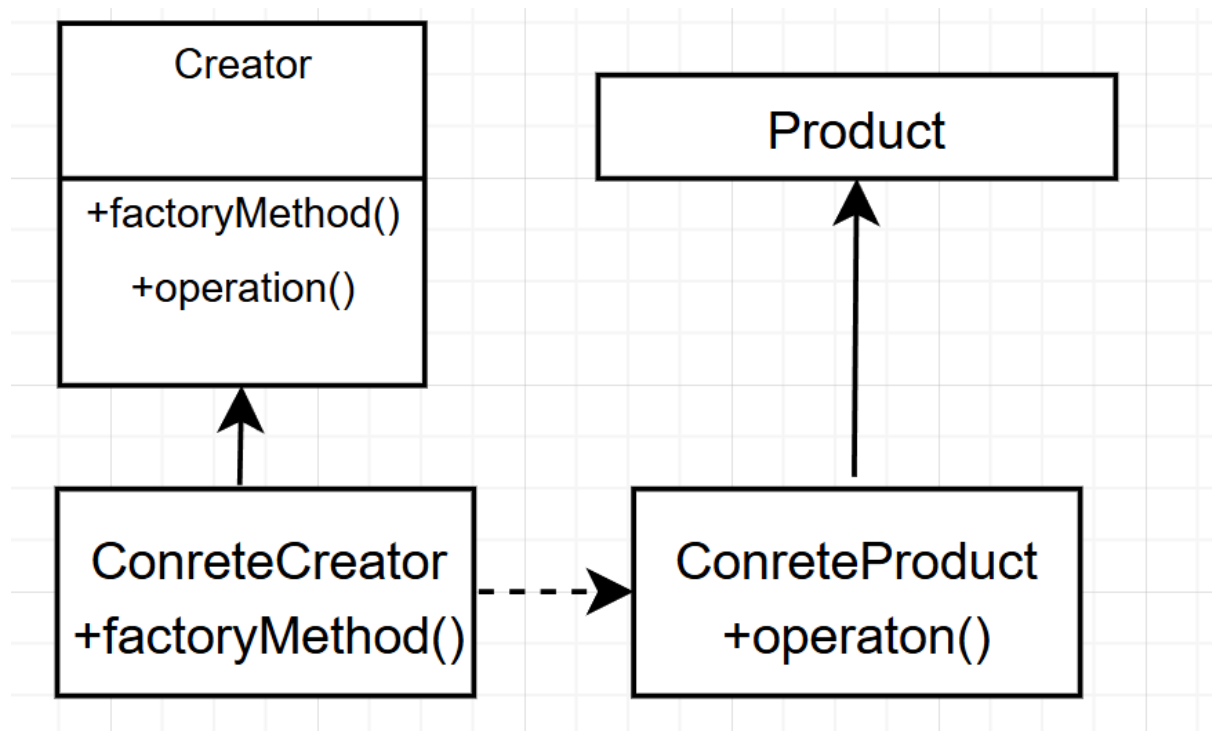
3. Які класи входять у шаблон «Абстрактна фабрика» і яка між ними взаємодія?

До шаблону входить `AbstractFactory`, що визначає інтерфейси створення різних типів продуктів, `ConcreteFactory`, яка створює конкретні продукти, `AbstractProduct` як загальний інтерфейс для продуктів, `ConcreteProduct` як реалізація продуктів, а також `Client`, який працює лише з фабрикою і не знає про конкретні класи. Клієнт створює фабрику й отримує через неї узгоджені об'єкти.

4. Яке призначення шаблону «Фабричний метод»?

Його мета полягає в тому, щоб передати процес створення об'єктів підкласам, дозволяючи змінювати тип створюваних об'єктів без зміни коду клієнта.

5. Нарисуйте структуру шаблону «Фабричний метод».



6. Які класи входять у шаблон «Фабричний метод» і яка між ними взаємодія?

До шаблону входять **Product** як спільний інтерфейс, **ConcreteProduct** як реалізація продукту, **Creator** із фабричним методом, що повертає **Product**, та **ConcreteCreator**, який перевизначає цей метод для створення потрібного типу продукту.

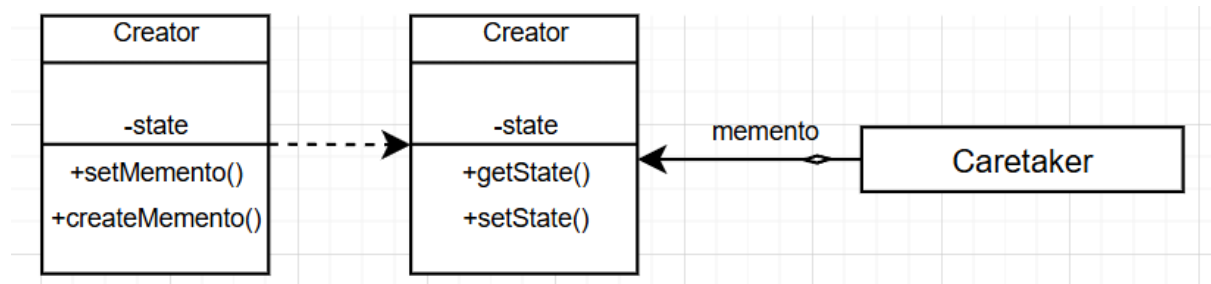
7. Чим відрізняється «Абстрактна фабрика» від «Фабричного методу»?

Абстрактна фабрика створює ціле сімейство пов'язаних об'єктів та містить кілька фабричних методів, а фабричний метод зосереджується на створенні одного типу об'єкта. Через це «Абстрактна фабрика» є складнішим шаблоном.

8. Яке призначення шаблону «Знімок»?

Його призначення - зберегти та відновити попередній стан об'єкта, не порушуючи інкапсуляції та не надаючи прямого доступу до внутрішніх даних.

9. Нарисуйте структуру шаблону «Знімок».



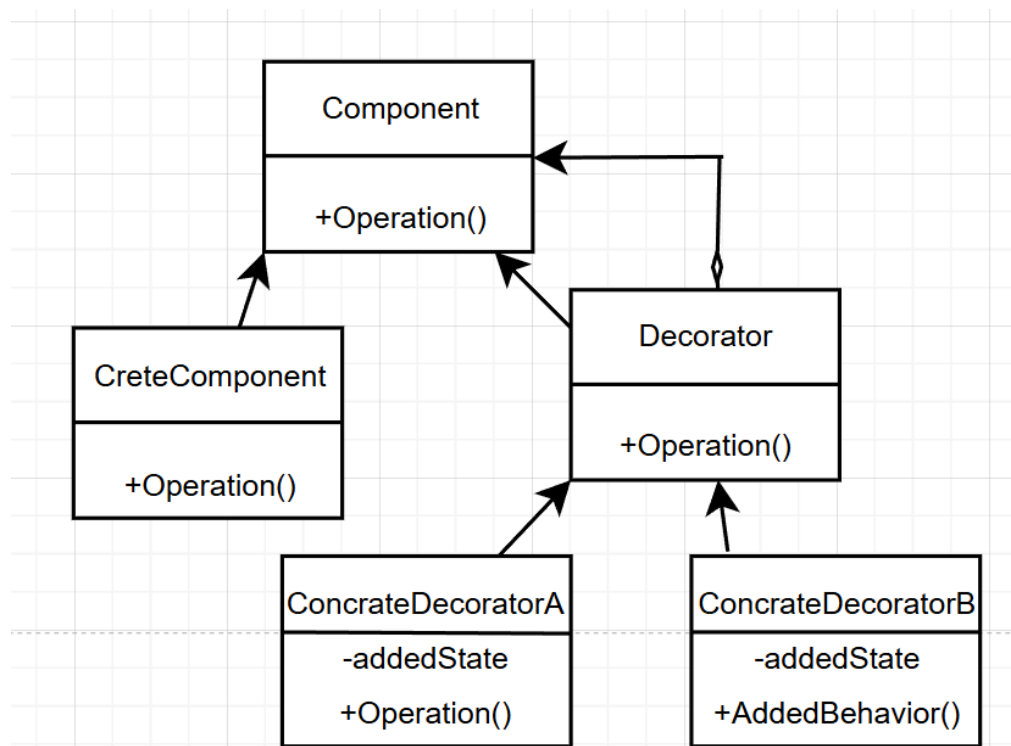
10. Які класи входять у шаблон «Знімок» і яка між ними взаємодія?

У шаблоні є Originator, що створює стан і формує Memento, Memento, який зберігає цей стан, та Caretaker, що відповідає за збереження й повернення знімків. Originator створює Memento, Caretaker його зберігає, а при необхідності повертає назад для відновлення стану.

11. Яке призначення шаблону «Декоратор»?

Мета - динамічно розширювати поведінку об'єктів, не змінюючи їхніх класів і не зачіпаючи існуючу структуру наслідування.

12. Нарисуйте структуру шаблону «Декоратор».



13. Які класи входять у шаблон «Декоратор» і яка між ними взаємодія?

У шаблон входить Component як спільний інтерфейс, ConcreteComponent як базовий об'єкт, Decorator, що містить посилання на компонент і делегує йому виклики, та ConcreteDecorator, який додає нову поведінку перед або після делегування.

14. Які є обмеження використання шаблону «Декоратор»?

До недоліків належать велика кількість дрібних класів, що ускладнює структуру, труднощі з відлагодженням при значній кількості обгортки, неможливість змінювати поведінку вже побудованого ланцюга без його повного відновлення та неефективність у випадках, коли потрібно глобально змінити поведінку для всіх об'єктів певного класу.

